Ejercicio 1: createGraph(List, List)

- **Objetivo:** Crear un grafo representado por un arreglo de listas de adyacencia Graph = Array (n, LinkedList()).
- **Solución algorítmica:**
    1. Contar la cantidad de vértices en la lista de vértices para definir el tamaño n del arreglo.
    2. Inicializar un arreglo (o estructura equivalente) de tamaño n donde cada posición contendrá una lista enlazada vacía (LinkedList()).
    3. Mapear cada vértice a un índice numérico (si no lo están ya) para facilitar el acceso directo en el arreglo.
    4. Recorrer la lista de aristas. Por cada arista que conecta el vértice u con el vértice v:
        - Agregar v a la lista enlazada ubicada en la posición u del arreglo.
        - Como es un grafo simple no dirigido (por lo general), agregar simétricamente u a la lista enlazada ubicada en la posición v.

Ejercicio 2: existPath(Grafo, v1, v2)

- **Objetivo:** Determinar si existe un camino entre dos vértices v_1 y v_2.
- **Solución algorítmica:**
    1. Utilizar un recorrido en profundidad (DFS) o en anchura (BFS) partiendo desde el vértice v_1.
    2. Mantener una estructura auxiliar de visitados (un arreglo de booleanos o una lista) para evitar ciclos y estados repetidos.
    3. Durante el recorrido, si en algún momento visitamos el vértice v_2, retornar inmediatamente True.
    4. Si el recorrido finaliza por completo sin haber encontrado a v_2, retornar False.

Ejercicio 3: isConnected(Grafo)

- **Objetivo:** Verificar si el grafo es conexo (existe camino entre todo par de vértices).
- **Solución algorítmica:**
    1. Seleccionar un vértice de inicio arbitrario (por ejemplo, el primer vértice del grafo).
    2. Ejecutar un recorrido (DFS o BFS) desde ese vértice para marcar todos los nodos alcanzables.
    3. Al finalizar el recorrido, verificar la estructura de visitados: si hay algún vértice del grafo que no haya sido marcado como visitado, significa que el grafo está desconectado y se retorna False.
    4. Si todos los vértices fueron visitados, retornar True.

Ejercicio 4: isTree(Grafo)

- **Objetivo:** Determinar si el grafo es un árbol.
- **Solución algorítmica:** Un grafo no dirigido es un árbol si es conexo y no tiene ciclos (o equivalentemente, tiene V vértices y $V - 1$ aristas). Aplicando la lógica de recorrido:

1. Verificar la condición estructural básica: el número de aristas debe ser exactamente igual al número de vértices menos uno. Si no se cumple, retornar False.
2. Ejecutar una verificación de conexidad (como en el Ejercicio 3) para asegurar que no existan componentes aisladas.
3. Durante el recorrido de exploración, utilizar un mecanismo de control de padres (o componentes conexas) para garantizar que no se detecte ninguna arista de retroceso hacia un nodo ya visitado que no sea su padre directo (lo que indicaría un ciclo). Si se detecta un ciclo, retornar False. Si pasa ambas pruebas, retornar True.

Ejercicio 5: isComplete(Grafo)

- **Objetivo:** Determinar si el grafo es completo (existe una arista entre todo par de vértices).
- **Solución algorítmica:**
    1. Sea V la cantidad total de vértices del grafo.
    2. Recorrer cada lista de adyacencia del arreglo.
    3. Contar el grado (número de vecinos) de cada vértice. En un grafo completo, cada vértice debe estar conectado exactamente a $V - 1$ vértices.
    4. Si al menos un vértice tiene un grado menor a $V - 1$, retornar False.
    5. Si todos los vértices cumplen con tener un grado de $V - 1$, retornar True.

Ejercicio 6: convertTree(Grafo)

- **Objetivo:** Encontrar una lista de aristas cuya eliminación convierta el grafo en un árbol.
- **Solución algorítmica:**
    1. Inicializar una lista vacía para almacenar las aristas a eliminar.
    2. Recorrer el grafo utilizando un algoritmo de búsqueda (DFS o BFS) para construir un árbol de cobertura (spanning tree) y registrar las aristas de árbol frente a las aristas de retroceso (que son las que cierran ciclos).
    3. Cada vez que el algoritmo de exploración detecte una arista que conduce a un vértice ya visitado (y que no es el nodo padre inmediato en el árbol de búsqueda), dicha arista forma parte de un ciclo.
    4. Agregar esa arista a la lista de elementos a eliminar y continuar hasta asegurar que el grafo subyacente quede libre de ciclos y conexo. Retornar la lista de aristas recolectadas.

Ejercicio 7: countConnections(Grafo)

- **Objetivo:** Calcular el número de componentes conexas del grafo.
- **Solución algorítmica:**
    1. Inicializar un contador de componentes en 0 y una estructura de control de visitados para todos los vértices (inicialmente todos en falso).
    2. Recorrer con un ciclo todos los vértices del grafo desde el índice 0 hasta $n - 1$.
    3. Si se encuentra un vértice que **no** ha sido visitado:
        - Incrementar el contador de componentes en 1.
        - Lanzar un recorrido (DFS o BFS) desde ese vértice para marcar todos los nodos alcanzables de esa misma componente conexa.

4. Al terminar de revisar todos los vértices, retornar el valor acumulado del contador.

Ejercicio 8 y 9: convertToBFSTree(Grafo, v) / convertToDFSTree(Grafo, v)

- **Objetivo:** Convertir un grafo en un árbol BFS o DFS utilizando un vértice raíz v.
- **Solución algorítmica:**
    1. Inicializar una nueva estructura de Lista de Adyacencia vacía que representará el árbol resultante.
    2. Utilizar una cola (para BFS) o una pila/recursión (para DFS) inicializada con el vértice raíz v, junto con un registro de visitados.
    3. Mientras la estructura auxiliar no esté vacía, extraer el vértice actual.
    4. Al descubrir vecinos no visitados desde el vértice actual, añadir formalmente la arista correspondiente dentro de la nueva estructura de lista de adyacencia del árbol y marcar el vecino como visitado.
    5. Retornar la nueva representación en forma de lista de adyacencia.

Ejercicio 10: bestRoad(Grafo, v1, v2)

- **Objetivo:** Encontrar el camino más corto (en cantidad de aristas) entre v_1 y v_2.
- **Solución algorítmica:**
    1. Dado que se busca el camino más corto en un grafo no ponderado, utilizar un recorrido en anchura (BFS).
    2. Mantener una estructura auxiliar de rastreo de predecesores (un arreglo o diccionario donde predecesor[nodo] indica de qué nodo provino al ser visitado).
    3. Inicializar una cola con v_1 y marcarlo como visitado.
    4. Realizar el BFS hasta encontrar el vértice v_2 o vaciar la cola.
    5. Si se encuentra v_2, reconstruir el camino yendo hacia atrás desde v_2 utilizando el arreglo de predecesores hasta llegar a v_1, y luego invertir el orden para que inicie en v_1 y termine en v_2. Si la cola se vacía sin alcanzar v_2, retornar una lista vacía.

Ejercicio 11: isBipartite(Grafo)

- **Objetivo:** Determinar si un grafo es bipartito (sin ciclos de longitud impar).
- **Solución algorítmica:**
    1. Utilizar un esquema de coloración de vértices (por ejemplo, dos colores: 0 y 1, o un arreglo de estados: no_coloreado, color_A, color_B).
    2. Recorrer los vértices del grafo (manejando posibles componentes desconexas). Para cada componente no coloreada, asignar un color inicial (ej. color_A) a su raíz.
    3. Realizar un recorrido (BFS o DFS) propagando los colores de forma alternada: si un nodo tiene color_A, todos sus vecinos adyacentes deben tener obligatoriamente color_B.
    4. Si durante el recorrido se descubre un vecino ya visitado que tiene **el mismo color** que el nodo actual, se ha encontrado un conflicto (un ciclo de longitud impar), por lo que se debe retornar False.
    5. Si se logra colorear todo el grafo sin contradicciones, retornar True.

Ejercicio 12 y 13: Demostraciones teóricas

- **Ejercicio 12:** Si a un árbol se le añade una arista entre cualquier par de vértices existentes, se conecta un par que ya poseía un camino único a través de las aristas previas del árbol, cerrando por ende **exactamente un ciclo**. Al haber un ciclo, deja de cumplir la propiedad fundamental de los árboles (ser acíclicos).
- **Ejercicio 13:** En un árbol BFS, los niveles representan la distancia mínima en aristas desde la raíz. Si se toma una arista (u, v) que no pertenece al árbol BFS, por propiedades de la exploración en anchura, sus niveles correspondientes no pueden saltar más de una capa de diferencia, ya que de haber estado a una distancia mayor, el nodo habría sido descubierto antes por el nivel intermedio adyacente. Por lo tanto, sus niveles difieren a lo sumo en 1.

Ejercicio 14: PRIM(Grafo)

- **Objetivo:** Implementar Prim usando Matriz de Adyacencia.
- **Solución algorítmica:**
    1. Partir de un vértice inicial arbitrario para agregarlo al conjunto de vértices visitados/incluidos en el árbol.
    2. Mantener un registro del costo mínimo para conectar cada vértice externo con el árbol actual (similar a un arreglo de distancias o pesos mínimos).
    3. En un bucle que se repita hasta incluir todos los vértices:
        - Buscar entre los vértices no incluidos aquel que posea la arista de menor costo (peso mínimo) que lo conecte con el árbol actual.
        - Añadir dicho vértice al conjunto del árbol y registrar la arista seleccionada.
        - Actualizar los costos mínimos de los vecinos restantes del vértice recién incorporado.
    4. Retornar el conjunto de aristas que conforman el árbol abarcador de costo mínimo.

Ejercicio 15: KRUSKAL(Grafo)

- **Objetivo:** Implementar Kruskal usando Matriz de Adyacencia.
- **Solución algorítmica:**
    1. Extraer todas las aristas del grafo a partir de la matriz de adyacencia y ordenarlas de menor a mayor según su costo/peso.
    2. Inicializar una estructura de conjuntos disjuntos (*Disjoint-Set* / *Union-Find*) donde cada vértice comience en su propia componente independiente.
    3. Iterar a través de la lista ordenada de aristas:
        - Para cada arista (u, v), verificar mediante la estructura de conjuntos disjuntos si u y v pertenecen a la misma componente conexa.
        - Si **no** pertenecen a la misma componente, significa que seleccionarla no formará un ciclo: añadir la arista al árbol abarcador resultante y fusionar (*Union*) los conjuntos de u y v.
    4. Detener el proceso cuando el árbol contenga exactamente $V - 1$ aristas y retornar el resultado.

- **Ejercicio 16, 17 y 18 (Demostraciones):** Se basan en la propiedad del corte y la optimalidad de los árbeles abarcadores. Al añadir o remover aristas evaluando los pesos mínimos de los cortes definidos, se demuestra que la estructura de costo mínimo se mantiene inalterada porque cualquier intercambio por una arista de mayor costo violaría la minimalidad.
- **Ejercicio 19 (Modificaciones a Prim):**
    - *Costo máximo:* Invertir la condición de selección de la arista mínima, buscando en cada paso la arista de **mayor** peso disponible hacia fuera del árbol.
    - *Árbol de recubrimiento cualquiera:* Modificar la función de costo para que devuelva un valor constante y neutro para cualquier arista válida, haciendo que Prim elija arbitrariamente la primera conexión disponible sin importar el peso.
    - *Forzar un conjunto de aristas E:* Pre-procesar el grafo antes de ejecutar Prim marcando los costos de las aristas pertenecientes a E con un valor de cero o negativo infinito para forzar su inclusión prioritaria inmediata en el árbol resultante.
- **Ejercicio 20:**
    - **Solución algorítmica:** Dado que todas las aristas tienen el mismo costo en un grafo conexo no ponderado, cualquier árbol abarcador requiere la misma cantidad de aristas ($V - 1$). Mediante un análisis de los bloques biconexos o puentes del grafo (utilizando matrices de adyacencia en tiempo $O(V^2)$), se identifican aquellas aristas que actúan como puentes obligatorios (cuya remoción desconecta el grafo). Las aristas que pertenezcan a estos puentes se marcan con 1 en la matriz $M[u, v]$, ya que deben estar obligatoriamente en todo AACM; el resto se marca con 0.

Ejercicio 21: shortestPath(Grafo, s, v) (Dijkstra con Matriz de Adyacencia)

- **Objetivo:** Encontrar el camino más corto ponderado desde un origen s hasta un destino v.
- **Solución algorítmica:**
    1. Inicializar un arreglo de distancias con valor infinito para todos los vértices, excepto para el vértice de origen s que tendrá distancia 0.
    2. Inicializar un arreglo de predecesores para reconstruir el camino posteriormente.
    3. Mantener un registro de vértices visitados/procesados.
    4. Realizar un ciclo de V iteraciones:
        - Seleccionar el vértice no visitado u que tenga la menor distancia acumulada actual.
        - Si la distancia de u es infinita o si $u == v$, se puede optimizar o finalizar la búsqueda.
        - Marcar a u como visitado.
        - Para cada vecino adyacente w de u (consultando la matriz de adyacencia): calcular la distancia tentativa pasando por u. Si esta distancia es menor que la registrada actualmente en el arreglo de distancias para w, actualizar la distancia y asignar a u como el nuevo predecesor de w.

5. Si al finalizar el nodo v tiene distancia infinita, retornar NONE. De lo contrario, reconstruir la lista de vértices desde el predecesor de v hasta s y retornar la lista ordenada.

Ejercicio 22: Variante con función de costos multiplicativa

- **a) Principio de optimalidad:** Se cumple de forma análoga. Si el costo de un camino se define como el producto de los costos de sus aristas y todos los costos son positivos, cualquier sub-camino de un camino mínimo también debe ser el camino de menor producto entre sus extremos respectivos; de lo contrario, podría sustituirse por uno menor reduciendo el producto total.
- **b) Condición de existencia:** Debe existir un camino físico conexionado entre los vértices y los valores de las aristas no deben generar comportamientos anómalos (como ciclos de absorción si los valores fueran menores a 1, aunque el enunciado aclara que $C(v, w) > 0$).
- **c) Por qué falla Dijkstra clásico:** Dijkstra asume una función de costos aditiva donde los acumulados crecen o se mantienen estables de forma monótona ($dist + peso \geq dist$). Con una función multiplicativa donde los pesos son estrictamente mayores que 0, el comportamiento del producto puede alterar las monotonicidades esperadas si existen escalas fraccionarias o rangos cruzados.
- **d) Algoritmo alternativo:** Aplicar una transformación logarítmica a los costos. Si $C(p) = C(v_0,v_1) * \dots * C(v_{k-1}, v_k)$, aplicar logaritmos convierte la multiplicación en una suma: $\log(C(p)) = \log(C(v_0,v_1)) + \dots + \log(C(v_{k-1},v_k))$. Con esta transformación, el problema se reduce exactamente al algoritmo de Dijkstra estándar o Bellman-Ford sobre costos sumativos.
- **e) Transformación con $C(v, w) > 1$:** Definiendo una nueva función de costos $C'(v, w) = \ln(C(v, w))$, como $C(v, w) > 1$, se garantiza que $\ln(C(v, w)) > 0$. Al ejecutar el algoritmo de Dijkstra tradicional usando C', los valores mínimos obtenidos corresponden directamente al logaritmo del costo mínimo original, bastando con aplicar la función exponencial inversa ($e^{resultado}$) para recuperar los costos de la función original C.